

Resource Collection Simulation

Objective

Create a terminal-based graphical simulation using Ratatui that simulates autonomous robots collecting resources on a procedurally generated map.

Requirements

Map Generation

- Generate a map with noise-based obstacles
- Populate the map with two types of resources:
 - Energy sources (represented as 'E')
 - Crystal deposits (represented as 'C')
- Resources should have random quantities (50-200 units each)

Robot Types

Implement two types of robots with distinct behaviors:

1. Scout Robots (represented as 'x')

- Explore the map randomly
- Discover and share resource locations
- Avoid known obstacles
- Cannot collect resources

2. Collector Robots (represented as 'o')

- Navigate to known resource locations
- Collect resources one unit at a time
- Return to base when carrying resources
- Unload resources at the central base

Base System

- Central base acts as:
 - Starting point for all robots
 - Resource storage and knowledge hub
 - Communication center for sharing discoveries
- Track total collected energy and crystals

Concurrent Architecture & Knowledge Management

- Each robot operates as an independent entity with limited local knowledge
- Robots start with no information about the map beyond their immediate surroundings
- Information sharing occurs through asynchronous communication mechanisms
- Key distributed behaviors to implement:
 - Scouts broadcast discovered resources and obstacles to other robots
 - Collectors communicate resource collection events for the base to update global state
 - Base system coordinates knowledge aggregation from all robot discoveries
 - Robots must synchronize their actions without blocking each other's operations

Technical Requirements

- Use Ratatui for terminal UI rendering
- Implement real-time simulation
- Handle user input (any key press exits)
- Use Rust's concurrency features for robot coordination
- Generate obstacles using Perlin noise

Visual Layout

Obstacles: 0 (light cyan)
Energy: E (green)
Crystals: C (light magenta)
Base: # (light green)
Scouts: x (red)
Collectors: o (magenta)
UI: Display collected resources counter

Success Criteria

- Robots autonomously navigate and avoid obstacles

- Scouts discover and share resource locations
- Collectors efficiently gather resources and return to base
- Real-time updates of resource collection progress
- Clean terminal rendering with proper color coding

Grading Rubric

Core Implementation (60 points)

- **Map Generation (10 points):** Noise-based obstacle generation, resource placement
- **Robot Behaviors (20 points):** Distinct scout and collector behaviors, pathfinding
- **Base System (10 points):** Resource storage, starting point functionality
- **Communication System (20 points):** Message passing, knowledge sharing, synchronization

Technical Quality (25 points)

- **Concurrent Architecture (10 points):** Independent robot entities, non-blocking operations
- **Ratatui Integration (8 points):** Real-time rendering, proper color coding
- **Code Quality (7 points):** Clean structure, proper error handling, documentation

Advanced Features (15 points)

- **Optimization (5 points):** Efficient pathfinding, resource allocation strategies
- **Robustness (5 points):** Handle edge cases, resource depletion, collision avoidance
- **User Experience (5 points):** Smooth simulation, clear visual feedback

Simulation de Collecte de Ressources

Objectif

Créer une simulation graphique en terminal utilisant Ratatui qui simule des robots autonomes collectant des ressources sur une carte générée procéduralement.

Exigences

Génération de Carte

- Générer une carte avec des obstacles basés sur du bruit
- Peupler la carte avec deux types de ressources :
 - Sources d'énergie (représentées par 'E')
 - Gisements de cristaux (représentés par 'C')
- Les ressources doivent avoir des quantités aléatoires (50-200 unités chacune)

Types de Robots

Implémenter deux types de robots avec des comportements distincts :

1. Robots Éclaireurs (représentés par 'x')

- Explorer la carte de manière aléatoire
- Découvrir et partager les emplacements de ressources
- Éviter les obstacles connus
- Ne peuvent pas collecter de ressources

2. Robots Collecteurs (représentés par 'o')

- Naviguer vers les emplacements de ressources connus
- Collecter les ressources une unité à la fois
- Retourner à la base en portant des ressources
- Décharger les ressources à la base centrale

Système de Base

- La base centrale agit comme :
 - Point de départ pour tous les robots
 - Centre de stockage de ressources et de connaissances
 - Centre de communication pour partager les découvertes
- Suivre le total d'énergie et de cristaux collectés

Architecture Concurrente et Gestion des Connaissances

- Chaque robot opère comme une entité indépendante avec des connaissances locales limitées
- Les robots commencent sans information sur la carte au-delà de leur environnement immédiat
- Le partage d'informations se fait par des mécanismes de communication asynchrone

- Comportements distribués clés à implémenter :
 - Les éclaireurs diffusent les ressources et obstacles découverts aux autres robots
 - Les collecteurs communiquent les événements de collecte pour que la base mette à jour l'état global
 - Le système de base coordonne l'agrégation des connaissances de toutes les découvertes robotiques
 - Les robots doivent synchroniser leurs actions sans bloquer les opérations des autres

Exigences Techniques

- Utiliser Ratatui pour le rendu de l'interface utilisateur terminal
- Implémenter une simulation en temps réel
- Gérer les entrées utilisateur (toute pression de touche quitte)
- Utiliser les fonctionnalités de concurrence de Rust pour la coordination des robots
- Générer les obstacles en utilisant le bruit de Perlin

Disposition Visuelle

Obstacles : **O** (cyan clair)
Énergie : **E** (vert)
Cristaux : **C** (magenta clair)
Base : **#** (**vert** clair)
Éclaireurs : **x** (rouge)
Collecteurs : **o** (magenta)
UI : **Afficher** le compteur de ressources collectées

Critères de Réussite

- Les robots naviguent de manière autonome et évitent les obstacles
- Les éclaireurs découvrent et partagent les emplacements de ressources
- Les collecteurs rassemblent efficacement les ressources et retournent à la base
- Mises à jour en temps réel du progrès de collecte des ressources
- Rendu terminal propre avec codage couleur approprié

Barème d'Évaluation

Implémentation de Base (60 points)

- **Génération de Carte (10 points)** : Génération d'obstacles basée sur le bruit, placement des ressources
- **Comportements des Robots (20 points)** : Comportements distincts d'éclaireur et collecteur, pathfinding
- **Système de Base (10 points)** : Stockage des ressources, fonctionnalité de point de départ
- **Système de Communication (20 points)** : Passage de messages, partage de connaissances, synchronisation

Qualité Technique (25 points)

- **Architecture Concurrente (10 points)** : Entités robotiques indépendantes, opérations non-bloquantes
- **Intégration Ratatui (8 points)** : Rendu en temps réel, codage couleur approprié
- **Qualité du Code (7 points)** : Structure propre, gestion d'erreurs appropriée, documentation

Fonctionnalités Avancées (15 points)

- **Optimisation (5 points)** : Pathfinding efficace, stratégies d'allocation des ressources
- **Robustesse (5 points)** : Gérer les cas limites, épuisement des ressources, évitement de collisions
- **Expérience Utilisateur (5 points)** : Simulation fluide, retour visuel clair